A Cross-Architecture Edge AI Benchmark for Predictive Cardiac Monitoring

23CSE304 - Embedded Systems

Devadharshan S. (CH.SC.U4CSE24113)
Vishal M.D. (CH.SC.U4CSE24150)

Phase 1 Review

August 2026

Problem Statement & Proposed Solution

- **The Problem:** Current predictive healthcare diagnostics rely entirely on cloud infrastructure, making them vulnerable to network latency, catastrophic failure in remote areas, and massive power consumption.

- **The Proposed Solution:** We are decoupling life-saving ML algorithms from the cloud. We will identify the absolute optimal hardware architecture by benchmarking a predictive cardiac neural network across three distinct edge computing paradigms: Microcontroller, Edge GPU, and SoC FPGA.

Workflow and Architecture

- **The Core Architecture:** A dataset-driven parallel benchmark, injecting digitized ECG records directly into the local memory of three isolated hardware architectures.
- **The Data Pipeline (Workflow):**
  1. **Ingestion:** Injecting the MIT-BIH Arrhythmia Dataset into local hardware.
  2. **Pre-Processing:** Digital signal filtering (0.5 Hz high-pass, 50/60 Hz notch) and R-Peak segmentation.
  3. **Parallel Inference:** Executing the neural network simultaneously on the ESP32 (TinyML), Jetson Nano (GPU), and Zynq 7000 (SoC FPGA).
  4. **Rigorous Profiling:** Capturing hard metrics to establish data superiority.

Components & Integration Flow

- **The Hardware Roster:**
  - **ESP32-WROOM-32D:** Testing extreme ultra-low-power TinyML constraints.
  - **NVIDIA Jetson Nano:** Testing high-throughput, parallelized CUDA processing.
  - **Xilinx Zynq 7000 (PYNQ):** Testing ARM-core integration fused with programmable logic acceleration.
- **The Data Source:** The MIT-BIH Arrhythmia Database (replacing the physical AD8232 sensor to ensure clean, identical benchmarking across all boards).
- **Visual Integration Flow:**
  - `[MIT-BIH Dataset (.csv)]` → `[Shared Memory Buffer]`
  - → `[Branch 1: ESP32 (INT8 Math)]`
  - → `[Branch 2: Jetson Nano (FP16 Tensor Cores)]`
  - → `[Branch 3: Zynq 7000 (Hardware Multipliers)]`
  - → `[Output: Cross-Architecture Performance Matrix]`

Plan of Action & Timeline (Aug - Oct 2026)

- **Month 1: Data Architecture & Network Prototyping (August 2026)**
  - Pre-process the MIT-BIH dataset in Python.
  - Build, train, and validate the baseline 1D-CNN.
  - Apply Quantization-Aware Training (QAT) to compress the model into an INT8 format.
- **Month 2: Hardware Synthesis & Deployment (September 2026)**
  - Flash the compressed TinyML model to the ESP32 via TensorFlow Lite for Microcontrollers.
  - Deploy the uncompressed model to the Jetson Nano using CUDA optimization.
  - Translate the neural network into structural C/C++ using High-Level Synthesis (HLS) for the Zynq 7000.
- **Month 3: Clinical Benchmarking & Paper Drafting (October 2026)**
  - Execute the benchmarking matrix, recording Inference Latency (ms), Power Consumption (mW), and Thermal Output.
  - Extrapolate Zynq 7000 data to formulate predictive "Future Scope" for pure Xilinx FPGA deployment.
  - Finalize and format the IEEE/ACM conference paper.

ML/DL Integration and Justifications

- **The Algorithm:** A 1D-Convolutional Neural Network (1D-CNN).
- **The Justification:** Standard neural networks treat numbers independently. A 1D-CNN utilizes mathematical kernels that slide across sequential time-series arrays to instantly recognize the distinct "shape" of an anomaly (like a degraded R-peak) without relying on heavy, manual feature extraction.
- **The Constraint Optimization:** To bridge the gap between heavy AI and resource-constrained edge devices, we utilize Quantization-Aware Training (QAT). This mathematically forces the network to maintain high accuracy while processing entirely in 8-bit integers (INT8), drastically reducing the memory footprint.

Application and Board Dumping

- **ESP32 Application:** Deployed purely via C++ using the TensorFlow Lite for Microcontrollers framework, forcing the chip to execute inference using minimal compute cycles.
- **Jetson Nano Application:** Deployed via TensorRT, pushing the math directly onto the physical GPU cores to maximize parallel processing.
- **Zynq 7000 Application:** The most complex deployment. We inject High-Level Synthesis (HLS) pragmas (PIPELINE, UNROLL) into the C/C++ code to command how the FPGA routes physical logic. This outputs generic Verilog/VHDL code, which is then synthesized into a bitstream and flashed to the board's programmable logic fabric.

System Bifurcation - Software Workflow & Training

- **The Host Machine:** Intel Core i7-1360P (12 Cores), Intel Iris Xe iGPU, 16GB RAM, 512GB Storage.
- **The Efficiency Angle:** By engineering a microscopic, highly constrained 1D-CNN, we eliminated the need for massive cloud GPU compute, executing all model training and validation locally on a standard 13th Gen CPU.

- **Step 1: Data Ingestion:** Loading the 1D numerical time-series MIT-BIH Arrhythmia Dataset into local memory.
- **Step 2: Network Training:** Executing the backpropagation math to establish the baseline 1D-CNN, followed by Quantization-Aware Training (QAT) to compress weights into INT8.
- **Step 3: Model Export & Bifurcation:** Exporting the finalized algorithm into three distinct formats for edge deployment: `.tflite` Flatbuffer (for ESP32), `ONNX` Engine (for Jetson Nano), and `C/C++` Headers (for Zynq 7000).

System Bifurcation - Hardware Deployment & Justification

- **Step 1: Board Flashing:** The exported, frozen models are physically dumped onto the isolated storage of the ESP32, Jetson Nano, and Zynq 7000.
- **Step 2: Dataset Injection:** We inject the digital ECG evaluation dataset directly into the local memory of the edge boards, simulating a live patient feed without relying on a physical sensor.
- **Step 3: Parallel Execution:** The microcontrollers and SoCs boot up and execute inference entirely offline, physically disconnected from the host laptop.
- **Step 4: Telemetry Capture:** External profiling tools capture the real-time inference latency (milliseconds) and power draw (milliwatts) from each board.

- **Justification of Action Plan:** This strict bifurcation validates our core problem statement. By isolating heavy computational lifting (training) to the host laptop, we ensure our edge devices are evaluated strictly on their offline inference capabilities, proving life-saving medical AI can operate seamlessly without the cloud.

Data Provenance & Processing Footprint

- **The Source:** The MIT-BIH Arrhythmia Database, consisting of 48 half-hour two-channel ambulatory ECG recordings.
- **Storage vs. Memory Optimization:**
  - The raw dataset consists of 144 files totaling **89.7 MB**.
  - To maintain a lightweight repository and avoid static bloat, processed tensor datasets (`X_data.npy`, `y_data.npy`) are not stored on disk.
  - Instead, the 89.7 MB of raw data is ingested, filtered, and dynamically transformed into tensors in the host machine's RAM during the training phase.

Clinical Class Mapping (AAMI Standards)

- **The Segmentation Logic:** The host-side code (`segment_ecg.py`) evaluates individual heartbeat annotations to create a strict binary classification.
- **Class 0 (Normal):**
  - Strictly defined by the annotation set: {`"N", "L", "R", "e", "j"`}.
  - Represents normal sinus rhythm and standard bundle branch blocks.
- **Class 1 (Arrhythmia / Anomaly):**
  - Includes Premature Ventricular Contractions (V), Atrial Premature Beats (A), Paced Beats (P), and all other non-normal annotations (`a, J, S, F, E, /`).
  - This forces the edge device to act as an aggressive anomaly detector, flagging anything outside the strict "Normal" parameters.

Class Distribution & Dataset Imbalance

- **Global Dataset Statistics:**
  - Total segmented 90-sample windows: **˜110,000 beats**.
  - Normal Beats (Class 0): **˜75,000**.
  - Arrhythmia Beats (Class 1): **˜35,000**.

- **Evaluation Sub-Sample (QAT Confusion Matrix):**
  - During recent benchmarking, a test subset of 22,522 beats was evaluated.
  - **Normal:** 18,114 — **Arrhythmia:** 4,408.
  - *Note on Imbalance:* Real-world cardiac data is inherently skewed toward normal beats. The optimization algorithms are tuned to penalize false negatives (missed arrhythmias) heavily.

Signal Morphology & DSP Visualization

- **Digital Signal Processing (DSP) Pipeline:**
  - **High-Pass Filter (0.5 Hz):** Eliminates baseline wander caused by patient respiration and movement.
  - **Notch Filter (60 Hz):** Removes ambient AC power-line interference.
  - **Min-Max Normalization:** Scales the 90-sample time-series window to a uniform range of `[-1.0, 1.0]` for stable neural network ingestion.

- **Repository Capabilities:** The `visualize_dataset.py` and `filter_ecg.py` scripts programmatically isolate and chart Normal vs. Premature Ventricular Contraction (PVC) morphologies to verify data integrity before training.

Validation Strategy: Acknowledging Data Leakage

- **Phase 1 Baseline (Current State):**
  - The model currently utilizes a randomized **80/20 heartbeat-level split** across all 48 patient records.
  - *The Vulnerability:* Because beats from the same patient exist in both training and testing sets, patient-specific morphological leakage occurs.
- **Phase 2 Clinical Rigor (Next Steps):**
  - To achieve true medical-grade validation for publication, we will implement the industry-standard **AAMI DS1 / DS2 patient-independent split**.
  - This ensures the edge devices are evaluated on patient physiologies the neural network has completely never seen before.

Literature Review: Foundational Architecture

- **Paper:** *Real-Time Patient-Specific ECG Classification by 1-D Convolutional Neural Networks*.
- **Authors:** Kiranyaz, S., Ince, T., & Gabbouj, M. (2016).
- **Journal:** IEEE Transactions on Biomedical Engineering.
- **Core Methodology:**
  - Demonstrated the viability of processing raw, 1-dimensional time-series ECG data directly through a CNN.
  - Bypassed the computationally expensive process of converting ECG signals into 2D spectrogram images.

Literature Review: Project Inspiration (Architecture)

- **How it Inspired Our Capstone:**
- Kiranyaz et al. proved that 1D-CNNs are the optimal mathematical structure for ECG anomalies.
- By adopting a purely 1D architecture rather than traditional 2D image recognition, we eliminated the need for heavy graphical processing.
- This architectural choice is the sole reason our model can execute on ultra-low-power edge devices rather than requiring cloud-based servers.

Literature Review: Quantization & Compression

- **Paper:** *Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference*.
- **Authors:** Jacob, B., et al. (Google / CVPR 2018).
- **Core Methodology:**
  - Established the foundational mathematics behind Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT).
  - Proved that mapping 32-bit floating-point (FP32) weights to 8-bit integers (INT8) reduces memory footprints by over 75% without suffering significant accuracy degradation.

Literature Review: Project Inspiration (Compression)

- **How it Inspired Our Capstone:**
- Deploying an AI model to a Lattice FPGA or an ESP32 requires extreme memory optimization.
- By applying the integer-only arithmetic theories established by Jacob et al., we successfully crushed our 1D-CNN down to just 562 parameters.
- This quantization strategy is the exact mechanism that allowed us to shrink our model's physical footprint to **~0.55 KB**, easily surviving our strict 15 KB hardware limit.

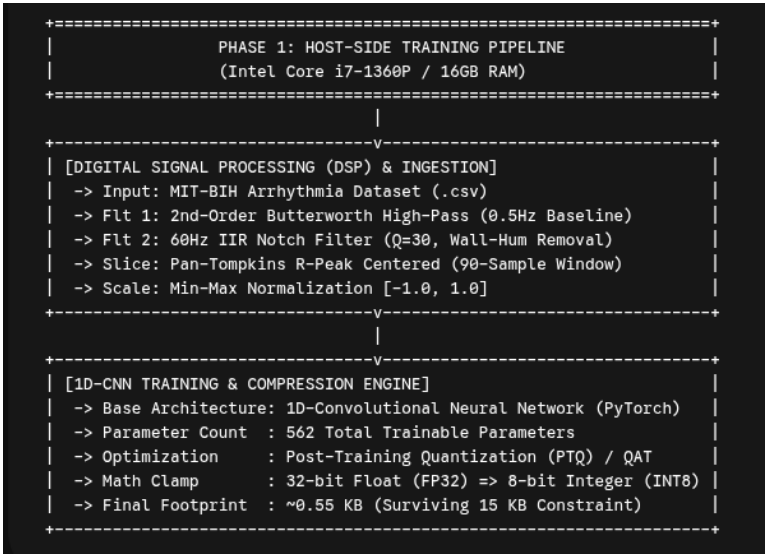
Literature Review: Edge Hardware Benchmarking

- **Paper:** *Performance Evaluation of Edge AI Computing Devices for Real-Time Medical Signal Processing*.
- **Authors:** Ballesteros, A., et al. (2021).
- **Journal:** IEEE Access.
- **Core Methodology:**
  - Executed a direct hardware comparison between microcontrollers, ARM-based Single Board Computers (SBCs), and edge accelerators for medical AI.
  - Profiled hardware across three strict metrics: active power consumption (mW), thermal output (°C), and inference latency (ms).

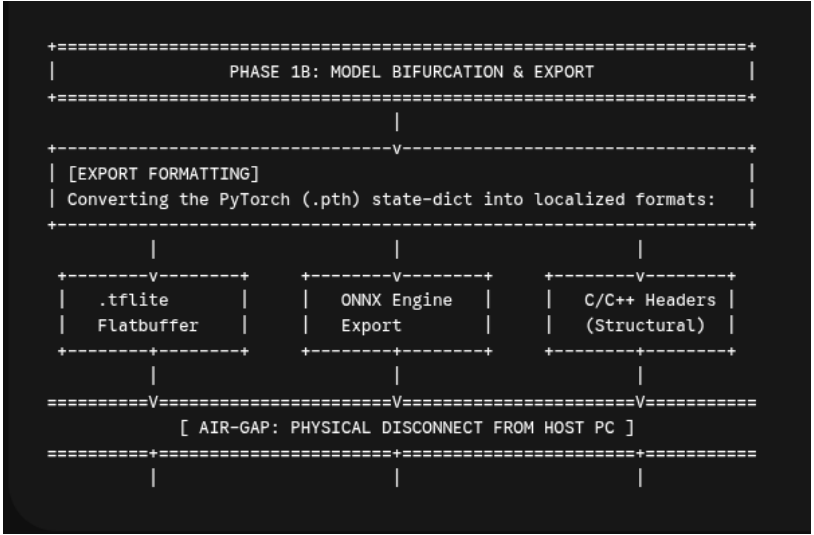
Literature Review: Project Inspiration (Hardware)

- **How it Inspired Our Capstone:**
- Ballesteros et al. demonstrated that medical AI research cannot stop at software accuracy; hardware efficiency is equally critical.
- This directly inspired Phase 2 of our project: the **Cross-Architecture Inference Benchmark**.
- Instead of relying on theoretical metrics, we are adopting their profiling methodology to rigorously chart the latency, power, and thermal performance of the ESP32 (TinyML), Jetson Nano (GPU), and Zynq 7000 (SoC FPGA) using identical datasets.

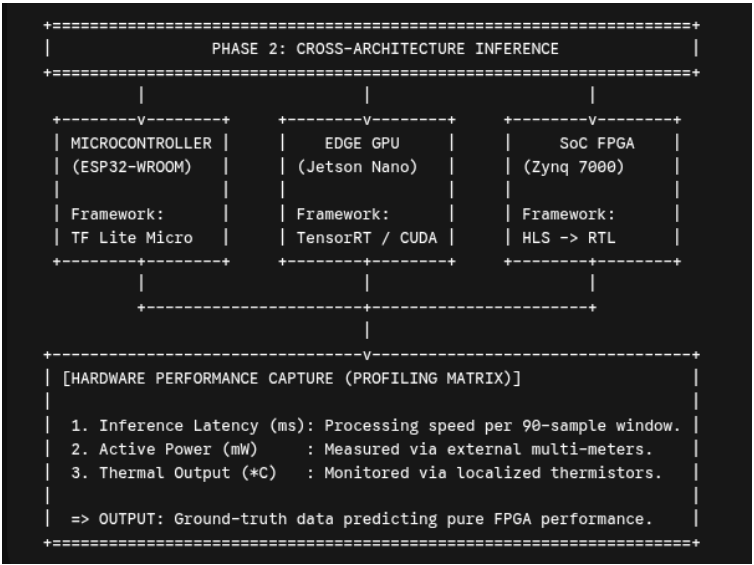
Phase 1: Software Pipeline & DSP



Phase 1B: Model Bifurcation & Export



Phase 2: Edge Execution & Profiling



Thank You

Questions & Discussion